inference-engine

A from-scratch LLM serving engine — Brief

Working explainer document (not published website content)

What this document is

A concise, accurate overview of `inference-engine`: a GPT-2 inference and serving engine built from scratch in Python and PyTorch. Every claim is drawn directly from the project's source code and README — nothing invented. See the companion `deep-dive.pdf` for exhaustive detail; this document trusts the reader to follow a well-structured summary.

1 What it is

`inference-engine` implements the core mechanisms of a modern LLM serving engine (in the spirit of vLLM) from scratch: an independent GPT-2 forward pass, a paged KV cache with prefix caching, a continuous-batching scheduler with preemption, speculative decoding, and an OpenAI-compatible HTTP API with SSE streaming — all running on CPU with GPT-2 family models (`gpt2`, `distilgpt2`). Hugging Face is used only to download pretrained weights; every serving mechanism is this project's own code.

Term: LLM serving engine

A system that runs a trained language model and answers many users' requests concurrently, efficiently sharing compute and memory between them — as opposed to a single-user inference script.

2 Architecture

- **server/app.py**: request validation (pydantic), SSE fan-out (one `asyncio.Queue` per sequence), routes `/v1/completions`, `/v1/models`, `/healthz`, `/metrics`.
- **AsyncEngine**: runs the blocking synchronous engine's `step()` on a worker thread so the asyncio event loop stays free; one lock serializes all engine mutations.
- **LLMEngine**: the synchronous core — scheduling, KV cache allocation, model execution, sampling, detokenization — callable directly from tests or benchmarks with no HTTP involved.
- **ModelRunner + GPT2**: the only layer that touches tensors; an independent GPT-2 forward implementation with paged attention.

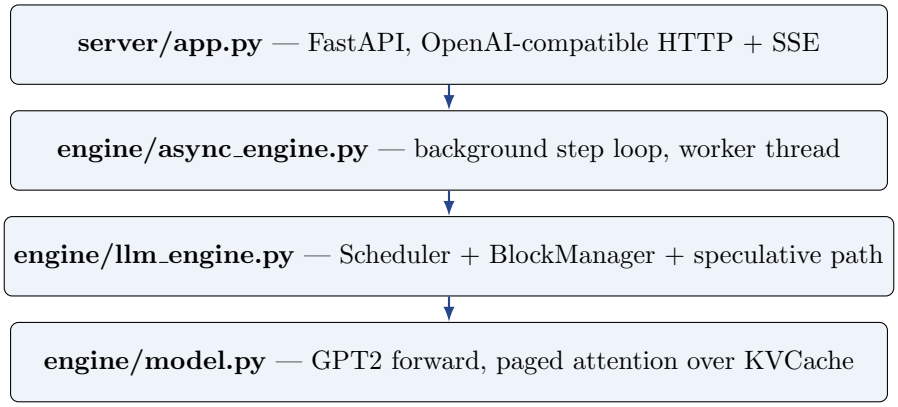

Figure 1: Four layers, top (HTTP) to bottom (tensors). Each layer only talks to the one directly below it: HTTP concerns never reach the scheduler, and scheduling logic never reaches tensor math.

3 The paged KV cache

Term: KV cache

Stored Key/Value vectors from each previous token's attention computation, reused instead of recomputed on every new token.

A naive cache reserves one contiguous region per sequence sized to its maximum possible length, wasting memory on tokens that may never be generated. This project borrows the operating-system idea of **paging**: KV memory is carved into fixed-size **blocks** (16 tokens by default), and each sequence owns a **block table** — a list of block ids — instead of a contiguous region. Allocation becomes on-demand, waste is bounded to one partial block per sequence, and two sequences can point at the same physical block.

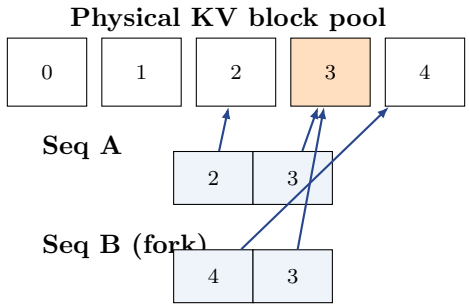

Figure 2: Two sequences with independent block tables sharing block 3 (reference count 2). A write into a shared block triggers copy-on-write: only the writer pays a copy, everyone else is untouched.

Prefix caching: once a block is completely full, its content is hashed (chained with every prior block's hash so identical windows in different contexts never collide). A new request whose prompt shares a hash chain with cached blocks reuses them instead of recomputing — freed hash-registered blocks sit in an LRU pool rather than being reclaimed immediately, specifically so this can happen. Rewriting a hashed block (stop- string truncation, or a rejected speculative draft) deregisters its hash first, closing a real correctness bug the project's own tests pin.

4 Continuous batching

Term: Continuous batching

The active batch is recomputed at every token boundary rather than fixed up front: whichever sequences are alive right now form the batch, so finished requests free their memory immediately and new requests join at the next step instead of waiting for a whole batch to complete.

The scheduler admits waiting prompts (**prefill**) while there is batch room and cache space, then checks — *cumulatively across the whole batch*, not per sequence — whether decode can take one more step. If not, the newest-arrived sequence is **preempted**: its blocks are freed and it is recomputed later (cheap, thanks to prefix caching). The cumulative check exists because the original per-sequence check let a batch that collectively did not fit pass every individual test and then crash — caught by the project's own scheduler tests.

5 Speculative decoding

Term: Speculative decoding

A small "draft" model proposes several tokens at once; the large "target" model verifies all of them in one forward pass, following the standard accept/reject rule (Leviathan et al., 2023) that keeps the output distributed exactly as if the target had sampled alone.

`distilgpt2` drafts k=4 tokens autoregressively; `gpt2` verifies all k+1 positions in one batched forward. Each draft token is accepted with probability $\min(1, p/q)$; the first rejection is replaced by sampling from the residual distribution and the rest of the round is rolled back (KV cache truncated, any rewritten prefix-cache hash deregistered).

Honest result: on this CPU, speculative decoding is *slower* (32.2 tok/s vs. 44.7 tok/s plain, 54% acceptance) because the draft model's own forward pass is not cheap enough relative to the target's to pay for itself. It is correct and stays in the codebase, but is explicitly not claimed as a CPU speedup — it would help where the target dominates draft cost (larger model, GPU).

6 Correctness: verified against a reference

The GPT-2 forward pass is tested against Hugging Face's own implementation (`tests/test_parity.py`): maximum absolute logit difference $< 10^{-3}$, measured $\approx 3 \times 10^{-5}$ on `distilgpt2` and $\approx 2 \times 10^{-4}$ on `gpt2`. This means every later bug found while building the cache/scheduler/batching machinery was guaranteed to be a systems bug, not a model-correctness bug. The full suite collects **55 tests** across 8 files (block manager, scheduler, engine, sampling, speculative, API, parity).

7 Benchmarks (CPU, Intel i7-1185G7, 8 threads)

Concurrency	Throughput (tok/s)	TTFT mean (s)
1	44.7	0.07
8	113.5	0.54
16	108.5	1.27

Continuous batching yields ≈ 2.5× more throughput from concurrency 1 to 8, then flattens as the 8-thread CPU saturates. Against sequential Hugging Face `generate()` (40.0 tok/s), the engine matches it one-at-a-time and is ≈ 2.8× faster batching 8 requests together. All numbers are relative, single-machine, CPU-only measurements; no vLLM/GPU comparison is claimed since none could be measured honestly without a GPU.

8 Known limitations (stated plainly, not hidden)

- Attention gathers each sequence's K/V into a padded dense tensor rather than using a truly fused paged kernel — memory savings are real, compute still touches padding.
- Prefills run one at a time (unbatched); ragged-prompt batching would cut TTFT under load but was not justified at this CPU scale.
- One coarse engine lock serializes all mutations — fine while the model forward dominates step time on CPU.
- Preemption always recomputes rather than swapping, because on CPU device and host memory are the same tier.
- Speculative decoding measured slower here, honestly reported rather than reframed.

9 Glossary

Block / paging
Fixed-size KV memory chunk addressed via a per-sequence block table, avoiding fragmentation and enabling sharing.

Copy-on-write
Sharing data until a write forces a private copy.

Prefill / decode
One-time full-prompt forward vs. the repeated one-token-at-a-time forward that follows.

Prefix caching
Reusing cached KV blocks for prompts sharing an identical leading token sequence.

TTFT
Time-to-first-token: latency until the first generated token reaches the client.

SSE
Server-Sent Events, the one-way HTTP streaming protocol used for token-by-token output.